# Spring Security

*Part 3 — Multiple Users and Database Integration*

## Introduction

Till now username and password were fixed in application.properties, and before that we were getting them from the console. Now what if we want multiple users? Because we do not want only one user for this application. Obviously we will want more users to sign up and use our application. In that case we need to do some configuration, so we will be storing multiple username and password combinations. We will start with in-memory and then slowly move towards having that data in a database. We will also look at how to secure that data because we do not want to store passwords as plain text on the server, so they should be encoded.

## Summary of All Lectures in a Single Flow

*Read this first. Running notes below are only for reference if needed.*

When someone tries to log in, Spring Security asks for credentials from a UserDetails object, which is returned by UserDetailsService. UserDetailsService is itself an interface, and Spring Boot automatically provides its own default implementation of it which looks for credentials in application.properties. If nothing is set there it generates a random password in the console.

But if we want credentials in memory we have to configure our own settings using the config file. So we create our own bean of UserDetailsService, create credentials of type UserDetails, and return them. But if we are dealing with a database then credentials will be coming dynamically from the database. In that case Spring Security provides us a DaoAuthenticationProvider. This provider does not talk to the database directly. It works with UserDetailsService which is the one that actually talks to the database. But this provider needs a UserDetailsService object to get the UserDetails from, so we have to pass our own UserDetailsService implementation class object here.

We have to create our own implementation class of UserDetailsService because we want things to work our way rather than the default Spring way. While implementing this interface we have to implement its only method loadUserByUsername which takes a username. Whenever we want some data from the database we need a repo layer in between to talk with the database. So we create a repo interface which will be extending JpaRepository because we are using JPA. We pass the entity class User and primary key type in its parameters. We also have to create the entity class User. Then we define a method findByUsername in the repo which will return a user by searching with username.

Now we inject the repo interface using @Autowired in our service implementation class and use this object to access its findByUsername method and assign the result to a variable of type User.

So now we have the user. But the loadUserByUsername method must return UserDetails, not a User object. UserDetails is also an interface so we have to create an implementation class for it

too. We create this class in the model package and name it UserPrincipal because in Spring Security when you are representing the currently logged in user you call them a principal. We could also name it UserDetailsImpl.

Since we are implementing an interface we have to implement all its methods. For now we return true in all the boolean methods and only deal with getUsername, getPassword, and getAuthorities. For getAuthorities we hardcode ROLE_USER for now. For getUsername and getPassword we need the actual user data. So we create a reference of the User class in UserPrincipal to store the incoming user object. We create a parameterized constructor to receive the User object passed from the service implementation class and assign it to our reference. Now we have the user object in UserPrincipal and we use user.getUsername() and user.getPassword() in the respective methods.

Coming back to the SecurityConfig file from where it all started. The provider now has the UserDetailsService object, which behind the scenes is the object of our implementation class, which is returning a UserDetails object, which behind the scenes is the object of UserPrincipal. Now we set the password encoder. For now we are not using any encoding so we set provider.setPasswordEncoder(NoOpPasswordEncoder.getInstance()). Then we return the provider object which will do the verification of the data passed to it and return the authenticated user if present, or throw user not found if not.

## Running Notes of All Lectures

*Detailed step by step notes — refer only if summary above is not enough*

## 1. Working With Multiple Users

Spring Security by default uses UserDetails to represent a user. UserDetailsService is the interface whose implementation looks for credentials in application.properties. We do not want this default behavior so we create our own bean in the config file.

By default User.builder().build() gives an empty object of UserDetails. We do not want to send an empty object. We want to send actual credentials that are also encoded. So we use withDefaultPasswordEncoder to create a user with encoded password.

### Why withDefaultPasswordEncoder is Deprecated

withDefaultPasswordEncoder encodes the password inline right there while creating the user object. It is deprecated because password encoding should happen properly before storing, not inline like this. It is fine for learning and testing but should never be used in a real production application. Later we will replace this with BCrypt.

### Creating Single User In Memory

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withDefaultPasswordEncoder()
            .username("Navin")
            .password("U@123")
            .roles("USER")
            .build();
```

```
    return new InMemoryUserDetailsManager(user);
}
```

### Creating Multiple Users In Memory

build() returns the object of UserDetails. We can create as many users as we want the same way and pass all of them to InMemoryUserDetailsManager.

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withDefaultPasswordEncoder()
            .username("Navin")
            .password("U@123")
            .roles("USER")
            .build();

    UserDetails admin = User.withDefaultPasswordEncoder()
            .username("Kiran")
            .password("K@456")
            .roles("ADMIN")
            .build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

## 2. Creating User Table and DB Properties

Now let us move to the database. We can use any SQL or NoSQL database like MySQL, PostgreSQL, or H2. We will go with PostgreSQL for now.

Make sure you have a database by the name telusko. Then create a table in it. We will create it by the name users because by default in PostgreSQL there is a built in table called user and we do not want to touch that.

### Create the Table

```
create table users (id integer primary key, username text, password text);
```

This creates 3 columns. Logically we should have one more column for roles because every user might have a different role like admin or user. But for now we will continue with 3 and add roles later.

### Check and Insert Data

```
select * from users;

insert into users(id, username, password)
values (1, 'kiran', 'n@789'), (2, 'harsh', 'h@123');

select * from users;
```

> *Passwords are stored as plain text for now. We will handle encoding later with BCrypt.*

### Add Database Settings in application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/telusko
spring.datasource.username=postgresql
```

```
spring.datasource.password=yourpassword
```

If using MySQL instead of PostgreSQL, replace postgresql with mysql in the URL and use port 3306 instead of 5432.

### Add Dependencies in pom.xml

Add two dependencies. One for the PostgreSQL driver because that is the database we are using. One for Spring Data JPA because that is how we will connect to the database. You can add these from the Maven repository or from the Spring Initializer when creating the project.

> *After adding database settings and dependencies, running the application will still fetch hardcoded values from the config file and not from the database. We need more classes and interfaces to change this, but this complex setup is done only once.*

## 3. What is Authentication Provider and Why

Every time a user sends username and password, Spring Security creates an Authentication object from those credentials. This Authentication object is then passed to an AuthenticationProvider which is responsible for verifying it. If the user exists it authenticates successfully. If not, it throws an exception.

By default Spring Security provides its own AuthenticationProvider behind the scenes. But now we want to change our provider. The new provider needs to connect with a database through DAO layers. For different types of authentication Spring Security provides different types of providers. Since we are working with a database we use DaoAuthenticationProvider.

DaoAuthenticationProvider is a class, not an interface, which means we can create an object of it directly. It extends AbstractUserDetailsAuthenticationProvider which implements AuthenticationProvider. So we can use it wherever an AuthenticationProvider is expected.

But just creating the object is not enough. The object has no idea which database we are using, how to represent a user, or what the user table name is. We have to tell it two things. First is the UserDetailsService which tells it how to fetch a user from the database. Second is the PasswordEncoder which tells it how to verify the password.

### Why We Set UserDetailsService in AuthenticationProvider

The AuthenticationProvider needs to know how to fetch a user by username from the database. UserDetailsService is the component that does exactly that. It has one method called loadUserByUsername which takes a username, goes to the database, finds the user, and returns it as a UserDetails object. By setting our custom UserDetailsService in the AuthenticationProvider we are telling it to use this to find the user when someone tries to log in.

### AuthenticationProvider Bean in SecurityConfig

```
@Autowired
private UserDetailsService userDetailsService;

@Bean
public AuthenticationProvider authProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
    provider.setUserDetailsService(userDetailsService);
    provider.setPasswordEncoder(NoOpPasswordEncoder.getInstance());
    return provider;
}
```

We also have to specify the password encoder. For now we do not want to use any encoding so we use NoOpPasswordEncoder.getInstance(). In future when we add BCrypt we will replace this line.

> *The in-memory UserDetailsService bean we created earlier needs to be commented out now because we want dynamic values coming from the database, not static hardcoded values.*

## 4. Package Structure

Before creating the service class let us arrange files into proper packages.

Create a package named model and move the Student class to it. Create a package named controllers and move all controllers to it. Create a package named service where we will create MyUserDetailsService. Create a package named dao where we will create UserRepo.

## 5. Creating UserDetailsService Implementation

Create a class called MyUserDetailsService in the service package. Mark it with @Service so Spring manages it. This class implements UserDetailsService interface. Spring will automatically use this class as the UserDetailsService because it is marked with @Service and implements the interface. The @Autowired in SecurityConfig will pick this up automatically.

We have to implement the only method of the interface which is loadUserByUsername. This method takes a username string.

In this method when we receive a username we have to hit the database to get the user details. To hit the database we need a repo layer. We create an object of UserRepo using @Autowired.

### UserRepo — Why Interface and Not Class

When using JPA we create an interface that extends JpaRepository. JPA automatically generates all standard database operations for us like find, save, and delete. We do not need to write any SQL ourselves. JPA generates the implementation behind the scenes. If we were using plain JDBC instead of JPA then we would write a class and write SQL queries manually. Since we are using JPA we create an interface.

### Service Class Code

```
@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepo repo;

    @Override
    public UserDetails loadUserByUsername(String username)
            throws UsernameNotFoundException {

        User user = repo.findByUsername(username);

        if (user == null) {
            throw new UsernameNotFoundException("User not found: " +
username);
        }
```

```
        return new UserPrincipal(user);
    }
}
```

## 6. User Repository

Create a dao package and create an interface UserRepo in it which extends JpaRepository. JpaRepository needs two things to be passed as type parameters. First is the entity class which maps to our database table. Second is the type of the primary key.

### What is an ORM Entity

JPA uses ORM which stands for Object Relational Mapping. It maps a Java class to a database table. Each field in the class maps to a column in the table. When we pass the entity class to JpaRepository, JPA knows which table to work with and how to convert rows from that table into Java objects and vice versa.

### Creating the User Entity Class

We do not have a User class yet so we create it in the model package. This class has the same properties as the columns we have in the table. We use @Data from Lombok for automatic getters and setters. We use @Entity to tell JPA this class maps to a database table. We use @Id to mark the primary key. We use @Table to specify the table name because our table is called users but our class is called User. If the class name and table name were the same we would not need @Table.

```
@Data
@Entity
@Table(name = "users")
public class User {
    @Id
    private int id;
    private String username;
    private String password;
}
```

### UserRepo Interface

We define a method findByUsername in the repo. JPA reads the method name and automatically generates the query to find a user by username. We do not need to write any SQL for this.

```
public interface UserRepo extends JpaRepository<User, Integer> {
    User findByUsername(String username);
}
```

## 7. Creating UserPrincipal — UserDetails Implementation

The loadUserByUsername method must return a UserDetails object but our repo returns a User object. UserDetails is an interface so we have to create an implementation class for it. We create this class in the model package and name it UserPrincipal. In Spring Security when you are representing the currently logged in user you call them a principal. We could also name it UserDetailsImpl.

Since we are implementing an interface we have to implement all its methods. For now we return true in all the boolean methods because we are not building that level of complexity yet. We only focus on three methods which are getUsername, getPassword, and getAuthorities.

For getAuthorities we hardcode ROLE_USER for now. If we want to handle this properly later we can add a roles column in the database table and return that value instead.

For getUsername and getPassword we need the actual user object data. So we create a reference of the User class in UserPrincipal to store the incoming user object. We create a parameterized constructor to receive the User object passed from the service class and assign it to our reference. Then we use storedUser.getUsername() and storedUser.getPassword() in the respective methods.

```
public class UserPrincipal implements UserDetails {

    private User storedUser;

    public UserPrincipal(User incomingUser) {
        this.storedUser = incomingUser;
    }

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return Collections.singleton(new SimpleGrantedAuthority("ROLE_USER"));
    }

    @Override
    public String getPassword() {
        return storedUser.getPassword();
    }

    @Override
    public String getUsername() {
        return storedUser.getUsername();
    }

    @Override
    public boolean isAccountNonExpired() { return true; }

    @Override
    public boolean isAccountNonLocked() { return true; }

    @Override
    public boolean isCredentialsNonExpired() { return true; }

    @Override
    public boolean isEnabled() { return true; }
}
```

## 8. The Complete Flow When Someone Logs In

Someone sends username and password. Spring Security creates an Authentication object. It passes this to DaoAuthenticationProvider. DaoAuthenticationProvider calls loadUserByUsername on our MyUserDetailsService. MyUserDetailsService uses UserRepo to query the database. If user is found it wraps the User object in a UserPrincipal object and returns it. DaoAuthenticationProvider uses the PasswordEncoder to compare the incoming

password with the stored one. If they match, authentication is successful. If user is not found, UsernameNotFoundException is thrown.

## 9. Final Package Structure

model — User, UserPrincipal

controllers — all controllers

service — MyUserDetailsService

dao — UserRepo

config — SecurityConfig

> ***Next up — BCrypt Password Encoding and User Registration.***